OnlineBankingAPI REST design

Oct 23, 2026 · @Kevin

Overview and conventions

OnlineBankingAPI is a JSON REST API under `/api/v1` that covers sign-in, accounts, transaction history, payees and transfers, backed by a relational database seeded with demo data. Every resource is scoped to the signed-in customer, and money only moves through one atomic, idempotent transfer operation.

Convention	Decision
Base URL	`https://{host}/api/v1` (version in the path)
Format	`application/json`, camelCase property names, UTF-8
Money	JSON number with 2 decimals (`1250.00`), stored as `decimal(19,4)`, C# `decimal`; never floating point
Currency	Single currency (USD) for the prototype; `currency` field returned on accounts for future use
Timestamps	ISO 8601 UTC with offset (`2026-10-21T14:05:00Z`), C# `DateTimeOffset`
Dates in filters	ISO date `YYYY-MM-DD`, inclusive, interpreted in the bank's time zone
Identifiers	Opaque GUIDs in URLs and bodies; account numbers shown masked (`****4821`)
Errors	RFC 9457 Problem Details (`application/problem+json`) with a stable `code`
Ownership	A resource the caller does not own returns `404`, never `403`, so ids cannot be probed
Docs	OpenAPI 3.1 document generated by ASP.NET Core and published at `/openapi/v1.json`

Authentication and session

Sign-in issues an opaque, server-side session rather than a self-contained JWT, because the requirement says signing out must end the session and a revocable server record does that immediately.

- **Session token.** 256-bit random value. The database stores only its SHA-256 hash in `Sessions`.

- **Transport.** Set as cookie `obk_session` (`HttpOnly`, `Secure`, `SameSite=Strict`, `Path=/`; site-wide since 2026-10-25 so the web front end's `proxy.ts` can pre-check it on page requests). Non-browser clients (tests, Postman) may send the same token as `Authorization: Bearer <token>`; the login response returns it in the body for that purpose.
- **Lifetime.** 15-minute sliding idle timeout, 8-hour absolute cap. Each authenticated request refreshes `LastSeenAt`.
- **Sign-out.** Sets `RevokedAt` on the session row and expires the cookie. Any later request with that token gets `401`.
- **Passwords.** Hashed with ASP.NET Core Identity's `PasswordHasher<T>` (PBKDF2). Five failed attempts lock the user for 15 minutes.
- **Protected endpoints.** Everything except `POST /auth/login` requires an authenticated session (global `[Authorize]` fallback policy).
- **Data isolation.** The customer id comes only from the session, never from the request. Every query filters on `CustomerId`, so another customer's account or payee id returns `404 not_found`.
- **CSRF.** `SameSite=Strict` plus JSON-only request bodies; bearer-token clients are not exposed.

Endpoints

`POST /api/v1/auth/login`

```
{ "username": "alice", "password": "Demo#2026" }
```

`200 OK` (also sets the cookie)

```
{
  "customer": { "id": "8f1c…", "displayName": "Alice Morgan", "username":
"alice" },
  "sessionToken": "q3Zp…",
  "expiresAt": "2026-10-23T08:32:00Z"
}
```

Bad credentials return `401 invalid_credentials` with the same message whether the username or the password was wrong. A locked account returns `423 account_locked`.

`POST /api/v1/auth/logout` returns `204 No Content`. It is safe to call twice.

`GET /api/v1/auth/me` returns the `customer` object above, or `401` when the session has ended. A client uses it to decide whether to show protected screens.

Endpoint catalogue

Twelve endpoints cover the four requirement areas; all paths are relative to `/api/v1` and all except login require a session.

Method	Path	Purpose	Req.
POST	`/auth/login`	Sign in, start a session	01
POST	`/auth/logout`	End the current session	01
GET	`/auth/me`	Current customer, session check	01
GET	`/accounts`	All of the customer's accounts with balances	02
GET	`/accounts/{accountId}`	One account	02
GET	`/accounts/{accountId}/transactions`	History: newest first, filter, search, paging, running balance	03
GET	`/accounts/{accountId}/transactions/{transactionId}`	One transaction	03
GET	`/payees`	The customer's saved payees	04
POST	`/transfers/quotes`	Validate a transfer and return the confirmation summary	04

Method	Path	Purpose	Req.
POST	`/transfers`	Execute a confirmed quote (requires `Idempotency-Key`)	04
GET	`/transfers/{transferId}`	Transfer result	04
GET	`/transfers`	Recent transfers, newest first	04

Payees are read-only in the prototype because the requirement uses a seeded list; `POST /payees` is a natural next step.

Accounts

`GET /accounts` returns every account the customer owns with its current balance, read from the account row that each transfer updates in the same database transaction as its ledger entries.

`200 OK`

```
{
  "items": [
    {
      "id": "3b7e…",
      "type": "checking",
      "name": "Everyday Checking",
      "maskedNumber": "****4821",
      "currency": "USD",
      "balance": 2450.75,
      "availableBalance": 2450.75,
      "asOf": "2026-10-23T00:30:12Z"
    },
    {
      "id": "c90a…",
      "type": "savings",
      "name": "High-Yield Savings",
      "maskedNumber": "****7730",
      "currency": "USD",
      "balance": 10400.00,
```

```
      "availableBalance": 10400.00,
      "asOf": "2026-10-23T00:30:12Z"
    }
  ],
  "totalBalance": 12850.75
}
```

- `type` is `checking` or `savings`.
- `availableBalance` equals `balance` in the prototype (no holds), kept so a client never needs to change when holds arrive.
- Accounts are ordered checking first, then by name.
- `GET /accounts/{accountId}` returns a single object of the same shape, or `404 not_found` if the account does not exist or belongs to someone else.
- Each account response carries an `ETag` header (the row version) so a client can cheaply re-poll with `If-None-Match` and receive `304`.

Transaction history

`GET /accounts/{accountId}/transactions` returns ledger entries newest first, with cursor paging, date and type filters, text search, and a running balance that is stored on each entry when it is posted.

Query parameters

Parameter	Type	Default	Rule
`from`	date	none	Inclusive start date (`2026-09-01`)
`to`	date	none	Inclusive end date; `from` after `to` is `400 invalid_date_range`
`type`	`debit` | `credit`	both	Direction filter
`q`	string, 2 to 100 chars	none	Case-insensitive contains-match on description, counterparty and note
`pageSize`	int 1 to 100	25	Items per page
`cursor`	string	none	Opaque value from the previous page's `nextCursor`

Filters combine with AND. Changing any filter starts a new cursor chain; a cursor reused with different filters returns `400 invalid_cursor`.

Response

```
{
  "accountId": "3b7e…",
  "items": [
    {
      "id": "e41d…",
      "postedAt": "2026-10-21T14:05:00Z",
      "description": "Transfer to High-Yield Savings",
      "counterparty": "High-Yield Savings ****7730",
      "type": "debit",
      "amount": 200.00,
      "signedAmount": -200.00,
      "runningBalance": 2450.75,
      "category": "transfer",
      "note": "October savings",
      "transferId": "7a02…"
    }
  ],
  "page": { "pageSize": 25, "nextCursor": "eyJwIjoi…", "hasMore": true },
  "totalCount": 312
}
```

- `amount` is always positive; `type` says debit or credit, and `signedAmount` saves the client the arithmetic.
- `category` is one of `transfer`, `payment`, `deposit`, `card`, `fee`, `interest`.
- `transferId` is present when the entry came from a transfer, linking the debit and its matching credit.

Ordering and paging

Entries sort by `(PostedAt DESC, Sequence DESC)`, where `Sequence` is a per-account, gap-free counter assigned at posting. The cursor is a base64url encoding of the last item's `(postedAt, sequence)` plus a hash of the filters. Keyset paging stays fast on long histories and does not skip or repeat rows when a new transfer posts while the customer is scrolling. `totalCount` is the count for the current filters, so a client can show "312 results".

Running balance

Each ledger entry stores `BalanceAfter`, written in the same transaction that updates the account balance under a row lock. The running balance is therefore the true account balance immediately after that entry, and it is unaffected by filters or search. A filtered view shows the real balance at each matching point, not a sum of the visible rows, so a client should label the column "Balance after".

Why stored rather than computed with a window function: it is correct across pages without scanning the whole history, and it never changes after posting because the ledger is append-only. The integrity check `BalanceAfter = previous BalanceAfter + SignedAmount` runs in tests and in a seed verification step.

`GET /accounts/{accountId}/transactions/{transactionId}` returns a single item of the same shape.

Payees

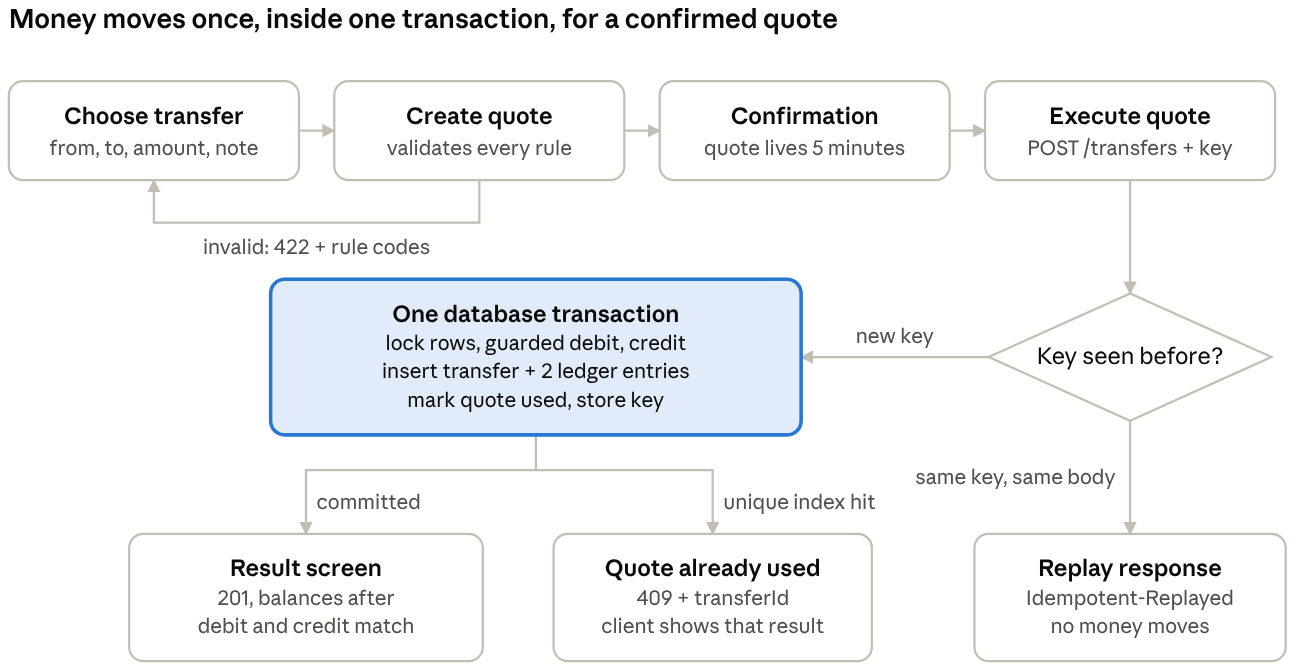
`GET /payees` returns the customer's saved payees from the seeded list, ordered by nickname.

```
{
  "items": [
    {
      "id": "51d9…",
      "nickname": "Landlord",
      "name": "Oakview Property Management",
      "bankName": "First Harbor Bank",
      "maskedAccountNumber": "****0912",
      "category": "rent"
    }
  ]
}
```

A payee is external to the bank, so a payment to a payee writes one debit on the customer's account and one credit on the bank's internal clearing account (see the schema). Full payee account numbers are never returned.

Transfers and confirmation

A transfer takes two calls: `POST /transfers/quotes` validates the request and returns the confirmation summary, then `POST /transfers` executes that quote exactly once. The server will not move money without a valid, unexpired, unused quote, so the confirmation step is enforced by the API rather than left to the client.



transfer flow · quote, confirm, execute, with idempotency outcomes

Only the highlighted transaction moves money; a replayed key or a reused quote returns the original result instead. The same key with a different body is refused with 422.

Step 1: create a quote

`POST /api/v1/transfers/quotes`

```
{
  "sourceAccountId": "3b7e…",
  "destination": { "kind": "account", "id": "c90a…" },
  "amount": 200.00,
  "note": "October savings"
}
```

`destination.kind` is `account` (one of the customer's own accounts) or `payee` (a saved payee id).

`201 Created`

```
{
  "quoteId": "b8e4…",
  "source": { "id": "3b7e…", "name": "Everyday Checking", "maskedNumber":
"****4821", "balance": 2650.75 },
  "destination": { "kind": "account", "id": "c90a…", "name": "High-Yield
Savings", "maskedNumber": "****7730" },
  "amount": 200.00,
  "fee": 0.00,
  "note": "October savings",
  "sourceBalanceAfter": 2450.75,
  "expiresAt": "2026-10-21T14:10:00Z"
}
```

The client shows this as the confirmation screen. A quote lives 5 minutes and stores the exact source, destination, amount and note, so the executed transfer cannot differ from what the customer confirmed. Creating a quote moves no money and places no hold.

Validation rules

The quote endpoint applies every rule below; the execute endpoint re-checks ownership, account status and funds inside its database transaction, because balances can change between confirm and submit.

Rule	Error code	HTTP
Amount missing, zero or negative	`amount_not_positive`	422
Amount has more than 2 decimal places	`amount_invalid_precision`	422
Source account not found or not owned	`not_found`	404
Destination account or payee not found or not owned	`destination_not_found`	422
Source and destination are the same account	`same_account`	422
Amount exceeds the source's available balance	`insufficient_funds`	422
Source or destination account is not active	`account_not_active`	422
Note longer than 140 characters or contains control characters	`note_invalid`	422

Several field errors are returned together in one response (see the error model) so the form can mark every problem at once.

Step 2: execute the quote

`POST /api/v1/transfers` with header `Idempotency-Key: 6f1d2c3e-…` (a UUID the client generates when it shows the confirmation screen)

```
{ "quoteId": "b8e4…" }
```

`201 Created`, `Location: /api/v1/transfers/7a02…`

```
{
  "id": "7a02…",
  "reference": "TRF-20261021-000123",
  "status": "completed",
  "source": { "id": "3b7e…", "name": "Everyday Checking", "maskedNumber":
"****4821", "balanceAfter": 2450.75 },
  "destination": { "kind": "account", "id": "c90a…", "name": "High-Yield
Savings", "maskedNumber": "****7730", "balanceAfter": 10400.00 },
  "amount": 200.00,
  "note": "October savings",
  "debitTransactionId": "e41d…",
  "creditTransactionId": "f20b…",
  "createdAt": "2026-10-21T14:05:00Z"
}
```

This is the result screen's data. `destination.balanceAfter` appears only for own-account transfers; a payee's balance is not ours to show. Execution is synchronous, so a successful response is always `completed`; any failure returns an error and moves nothing.

How execution stays atomic

One database transaction does all of the following, or none of it:

1. Lock the quote row and check it is unexpired and unused.
2. Lock both account rows with SELECT … FOR UPDATE in ascending id order, so two opposite transfers cannot deadlock.
3. Debit with a guarded update: `UPDATE accounts SET balance = balance - @amount WHERE id = @source AND balance >= @amount`. Zero rows updated means `insufficient_funds`.
4. Credit the destination account, or the bank's payee clearing account for a payee payment.
5. Insert the `Transfers` row and its two `LedgerEntries` (one debit, one credit, same `TransferId`, each with `BalanceAfter` and the next `Sequence`).
6. Mark the quote used (`Quotes.TransferId`, unique) and store the idempotency record with the response.

`GET /transfers/{transferId}` returns the same body, so a client that lost the response can show the result. `GET /transfers` lists recent transfers newest first with the same cursor paging as transactions.

Idempotency: every transfer happens once

Two independent guards stop a double click, retry or refresh from moving money twice: an `Idempotency-Key` header that replays the first response, and a one-use quote enforced by a unique index in the database.

Guard 1: Idempotency-Key

- Required on `POST /transfers`; missing or not a UUID returns `400 idempotency_key_required`.
- Scoped to the customer: the unique key is `(CustomerId, Key)`, so one customer's key can never collide with another's.
- The server stores a SHA-256 hash of the canonical request body, the response status and body, and the `TransferId`. Records are kept 24 hours.

Situation	Server behaviour
New key	Execute the transfer and store the response with it
Same key, same body, already completed	Return the stored response unchanged (same status, body, `Location`) plus header `Idempotent-Replayed: true`; no money moves
Same key, same body, first request still running	The second request waits on the unique index, then replays the committed response; if the wait exceeds 5 s it returns `409 request_in_progress` with `Retry-After: 1`
Same key, different body	`422 idempotency_key_reused`; nothing executes
First attempt failed (e.g. insufficient funds)	Nothing is stored, the key is free, and a retry is evaluated fresh

The idempotency row is inserted inside the same database transaction as the transfer, so a crash can never leave a key recorded without its transfer, or a transfer without its key.

Guard 2: one-use quote

`Quotes.TransferId` has a unique filtered index. If a client loses its key (for example on a full page refresh) and submits the same quote with a new key, the second insert fails and the API returns `409 quote_already_used` with the existing `transferId`, so the client can fetch and show the original result.

Client contract

1. Generate the key once, when the confirmation screen opens, and keep it with the quote id (in memory and `sessionStorage`).
2. Disable the Confirm button while the request is in flight.
3. On a timeout or network error, retry with the same key and body.
4. On `409 quote_already_used`, call `GET /transfers/{transferId}` and show that result.
5. Start a new quote and a new key only when the customer begins a new transfer.

Error model

Every error is an RFC 9457 Problem Details body with a stable machine-readable `code`, a human `detail` the client may show, and a `traceId` for support.

```
{
  "type": "https://api.onlinebanking.example/errors/validation_failed",
  "title": "The transfer could not be accepted.",
  "status": 422,
  "code": "validation_failed",
  "detail": "Fix the highlighted fields and try again.",
  "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
  "errors": [
    { "field": "amount", "code": "insufficient_funds", "message": "Amount is
more than the available balance of 150.25." },
    { "field": "note", "code": "note_invalid", "message": "Note must be 140
characters or fewer." }
  ]
}
```

When several rules fail, the top-level `code` is `validation_failed` and each problem appears in `errors[]`; when one rule fails, the top-level `code` is that rule's code and `errors[]` holds it too, so clients only ever read `errors[]` for field highlighting.

HTTP	Code	When
400	`bad_request`	Malformed JSON or wrong types
400	`invalid_date_range`, `invalid_cursor`	Bad history query
400	`idempotency_key_required`	Missing or malformed `Idempotency-Key`
401	`unauthenticated`	No session, expired or signed out
401	`invalid_credentials`	Wrong username or password
404	`not_found`	Resource does not exist or is not the caller's
409	`quote_already_used`	Quote executed earlier; body carries `transferId`
409	`request_in_progress`	Same key still executing
422	`validation_failed` and rule codes	Any transfer validation rule (see Transfers)
422	`quote_expired`	Quote older than 5 minutes; create a new one
422	`idempotency_key_reused`	Same key, different body
423	`account_locked`	Too many failed sign-ins
429	`rate_limited`	Over 10 logins per minute per IP, or 60 requests per minute per session
500	`internal_error`	Unexpected failure; no details leaked

Unhandled exceptions go through ASP.NET Core's `UseExceptionHandler` with `AddProblemDetails`, so stack traces never reach the client.

Relational schema

Eight tables in PostgreSQL 17, mapped with EF Core and the Npgsql provider. Money is `numeric(19,4)`, ids are `uuid`, and times are `timestamptz`. C# entities and this doc use PascalCase names; the `EFCore.NamingConventions` snake_case convention maps them to `accounts`, `ledger_entries`, `balance_after` and so on in the database.

The ledger is double-entry: every transfer writes exactly one debit and one credit with equal amounts. A payment to an external payee credits the bank's internal **Payee clearing** account, so the books always balance.

Customers

Column	Type	Constraints
Id	uuid	PK
Username	citext	Unique, so case-insensitive
PasswordHash	varchar(200)	Not null
DisplayName	varchar(100)	Not null
Email	varchar(200)	Null
FailedLoginCount	integer	Default 0
LockoutEndsAt	timestamptz	Null
CreatedAt	timestamptz	Not null

Sessions

Column	Type	Constraints
Id	uuid	PK
CustomerId	uuid	FK Customers
TokenHash	bytea	Unique (SHA-256 of the token, 32 bytes)
CreatedAt, LastSeenAt, ExpiresAt	timestamptz	Not null
RevokedAt	timestamptz	Null; set on sign-out
UserAgent	varchar(300)	Null

Accounts

Column	Type	Constraints
Id	uuid	PK
CustomerId	uuid	FK Customers; null only for internal accounts
Kind	varchar(10)	Check in (`checking` , `savings` , `clearing`)
Name	varchar(100)	Not null
AccountNumber	char(10)	Unique
Currency	char(3)	Default `USD`
Balance	numeric(19,4)	Check `Kind = 'clearing' OR Balance >= 0`
Status	varchar(10)	Check in (`active` , `frozen` , `closed`)
NextSequence	bigint	Next ledger sequence for this account
xmin	xid (system column)	Concurrency token via `IsRowVersion()` , used as the ETag

Payees

Column	Type	Constraints
Id	uuid	PK
CustomerId	uuid	FK Customers
Nickname	varchar(50)	Unique per customer
Name, BankName	varchar(100)	Not null
AccountNumber	varchar(17)	Not null; never returned in full
RoutingNumber	char(9)	Not null
Category	varchar(20)	e.g. `rent` , `utilities` , `family`

Quotes

Column	Type	Constraints
Id	uuid	PK
CustomerId	uuid	FK Customers
SourceAccountId	uuid	FK Accounts
DestinationAccountId	uuid	FK Accounts, null for payee
PayeeId	uuid	FK Payees, null for own account
Amount	numeric(19,4)	Check > 0
Note	varchar(140)	Null
CreatedAt, ExpiresAt	timestamptz	Not null
TransferId	uuid	FK Transfers; partial unique index `WHERE transfer_id IS NOT NULL`

Check: `num_nonnulls(destination_account_id, payee_id) = 1`, and `source_account_id <> destination_account_id`.

Transfers

Column	Type	Constraints
Id	uuid	PK
Reference	varchar(24)	Unique, e.g. `TRF-20261021-000123`
CustomerId	uuid	FK Customers
SourceAccountId	uuid	FK Accounts
DestinationAccountId	uuid	FK Accounts, null for payee
PayeeId	uuid	FK Payees, null for own account
Amount	numeric(19,4)	Check > 0
Note	varchar(140)	Null
Status	varchar(10)	`completed`

Column	Type	Constraints
CreatedAt	timestamptz	Not null

Same destination checks as Quotes. `Reference` takes its number from a sequence (`transfer_reference_seq`).

LedgerEntries (transactions)

Column	Type	Constraints
Id	uuid	PK
AccountId	uuid	FK Accounts
Sequence	bigint	Unique with AccountId
TransferId	uuid	FK Transfers, null for seeded non-transfer entries
PostedAt	timestamptz	Not null
Direction	char(1)	Check in (`D`, `C`)
Amount	numeric(19,4)	Check > 0
BalanceAfter	numeric(19,4)	Not null
Description	varchar(200)	Not null
Counterparty	varchar(100)	Null
Category	varchar(20)	`transfer`, `payment`, `deposit`, `card`, `fee`, `interest`
Note	varchar(140)	Null

Indexes:

- `(account_id, posted_at DESC, sequence DESC) INCLUDE (amount, direction, balance_after, description)` for history pages.
- `(transfer_id)` for matching the two legs of a transfer.
- A `pg_trgm` GIN index on `description`, `counterparty` and `note`, so the `q` search (`ILIKE '%term%'`) stays fast on long histories.

The table is append-only: the app's database role has INSERT and SELECT only, and corrections would be reversing entries.

IdempotencyRecords

Column	Type	Constraints
CustomerId	uuid	PK part 1, FK Customers
IdempotencyKey	uuid	PK part 2
RequestHash	bytea	Not null (SHA-256)
ResponseStatus	integer	Not null
ResponseBody	jsonb	Not null
TransferId	uuid	FK Transfers
CreatedAt, ExpiresAt	timestamptz	ExpiresAt = CreatedAt + 24 h

A background `IHostedService` deletes expired idempotency records, expired quotes and ended sessions every hour.

Seed data plan

Three demo customers with 180 days of deterministic history, generated by a `DbSeeder` class that runs at startup in the Development and Demo environments and produces the same data on every reset.

Username	Name	Accounts (closing balance)	Payees	Demo purpose
`alice`	Alice Morgan	Everyday Checking (2,650.75), High-Yield Savings (10,200.00)	Landlord, City Power, Mom, Gym	Main walkthrough; about 320 checking entries for paging and search
`ben`	Ben Okafor	Checking (42.10), Savings (1,500.00), Holiday Savings (300.00)	Phone Co, Water Utility, Sister	Insufficient-funds demo, three-account picker

Username	Name	Accounts (closing balance)	Payees	Demo purpose
`chloe`	Chloe Tan	Checking (5,010.00), Savings (0.00)	Daycare, Car Loan	Empty savings history, first transfer into savings

All demo users share the password `Demo#2026`, documented in the README as demo-only. The Payee clearing account (`customerId = null`) is created first.

How the history is generated

1. Fixed GUIDs for customers, accounts and payees (constants in `SeedIds`), so tests and demo scripts can reference them.
2. `System.Random(20261023)` drives amounts and dates, so every reseed is identical.
3. Per checking account: an opening deposit 180 days ago, salary credits on the 1st and 15th, monthly rent and utility payments to payees, and 3 to 6 card purchases a week drawn from a list of 30 merchant names (for search: "Coffee", "Grocer", "Fuel").
4. Per savings account: monthly interest credits and two or three own-account transfers from checking, each written as a real `Transfers` row with a matching debit and credit.
5. Entries are posted in time order through the same ledger service the API uses, so `Sequence`, `BalanceAfter` and `Accounts.Balance` are computed, never hand-typed. The closing balances in the table are targets: the generator adds a final adjusting deposit to land on them.
6. A verification step after seeding asserts, for every account, that `Balance` equals the last `BalanceAfter`, that each `BalanceAfter` follows from the previous one, and that every transfer has one debit and one credit of equal amount. Seeding fails loudly if any check fails.

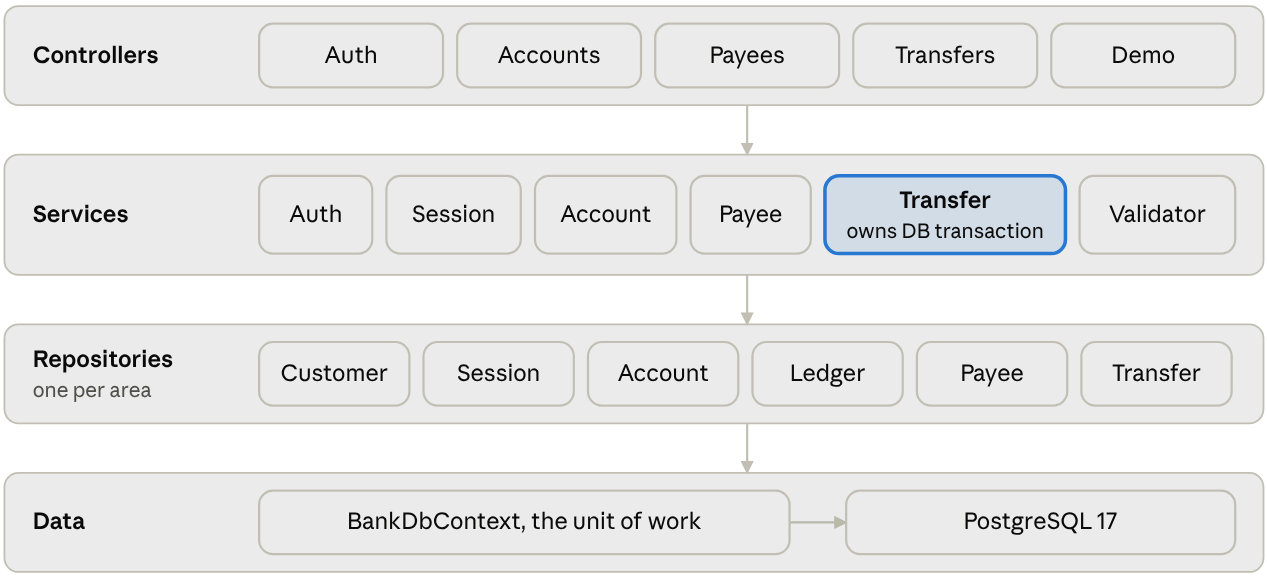
Reset: `dotnet run -- --reseed` drops and recreates the database, and `POST /api/v1/demo/reset` (Demo environment only, not in production builds) does the same for live demos.

Layered architecture

The code is split into three layers with one-way dependencies: controllers handle HTTP only, services hold the business rules and own the database transaction, and repositories hold all EF Core and SQL. Controllers never see `BankDbContext`, and services never write

a LINQ query.

Requests go down one layer at a time; only repositories touch EF Core



layers · controllers, services, repositories, data

Each layer calls only the layer below it. TransferService also uses BankDbContext directly, but only to begin, save and commit the transfer transaction.

What each layer may do

Layer	Responsible for	Never does
Controllers	Model binding, reading `CustomerId` from the session claims, calling one service method, setting status codes and headers (`Location`, `ETag`, `Idempotent-Replayed`, the session cookie)	Business rules, queries, `BankDbContext`
Services	Validation rules, ownership checks that span resources, the transfer transaction and row locks, mapping entities to response DTOs in `Contracts`, throwing `ApiProblem` with a rule code	LINQ, raw SQL, HTTP types (`HttpContext`, `IActionResult`)

Layer	Responsible for	Never does
Repositories	EF Core LINQ and raw SQL (`FOR UPDATE` , `nextval` , the guarded balance update), filtering every customer-owned query by `customerId`	`SaveChangesAsync` , transactions, business decisions, returning `IQueryable`
`BankDbContext`	Change tracking and the unit of work: services call `SaveChangesAsync` and commit once per operation	Being injected into a controller

Services

Service	Methods	Uses
`AuthService`	`LoginAsync`, `LogoutAsync`, `GetCurrentCustomerAsync` (lockout, password check)	`ICustomerRepository`, `SessionService`
`SessionService`	`CreateAsync`, `ValidateAsync`, `RevokeAsync` (token hashing, timeouts)	`ISessionRepository`
`AccountService`	`GetAccountsAsync`, `GetAccountAsync`, `GetHistoryAsync`, `GetTransactionAsync` (cursor decode and encode, filter rules)	`IAccountRepository`, `ILedgerRepository`
`PayeeService`	`GetPayeesAsync`	`IPayeeRepository`
`TransferService`	`CreateQuoteAsync`, `ExecuteAsync`, `GetAsync`, `ListAsync`; owns `BeginTransactionAsync`, lock order, the single `SaveChangesAsync` and `CommitAsync`	All four domain repositories, `TransferValidator`, `TimeProvider`
`TransferValidator`	Pure rule checks returning rule codes, no I/O	Nothing

`PayeeService` is thin today, but it keeps the rule that controllers only talk to services and gives payee management a home later. `LedgerService` folds into `ILedgerRepository`, because posting an entry is the guarded balance update plus an insert; the decision to post a matching debit and credit stays in `TransferService`.

Repositories

Each repository is a specific interface with intent-named methods, one per query the services need, with a single EF Core implementation. There is no generic `IRepository<T>`: it would either leak `IQueryable` or grow into a second, weaker copy of EF Core.

Interface	Methods
`IAccountRepository`	`GetAccountsForCustomerAsync(customerId)`, `GetOwnedAccountAsync(customerId, accountId)`, `GetClearingAccountIdAsync()`, `LockAccountsAsync(ids)` (ascending id, `FOR UPDATE`)
`ILedgerRepository`	`GetHistoryPageAsync(accountId, HistoryQuery)` returning the page and total count, `GetEntryAsync(accountId, entryId)`, `GetLegsForTransfersAsync(transferIds)`, `PostEntryAsync(LedgerPosting)`
`IPayeeRepository`	`GetPayeesForCustomerAsync(customerId)`, `GetOwnedPayeeAsync(customerId, payeeId)`
`ITransferRepository`	`AddQuote`, `GetQuoteForUpdateAsync(customerId, quoteId)`, `NextReferenceNumberAsync()`, `AddTransfer`, `GetTransferAsync(customerId, transferId)`, `GetTransfersPageAsync(customerId, after, take)`, `FindIdempotencyRecordAsync(customerId, key, now)`, `InsertIdempotencyRecordAsync(...)`, `CompleteIdempotencyRecordAsync(...)`, `DeleteExpiredIdempotencyRecordsAsync(now)`, `DeleteExpiredQuotesAsync(now)`
`ICustomerRepository`	`GetByUsernameAsync(username)`, `GetByIdAsync(customerId)`
`ISessionRepository`	`Add`, `GetActiveByTokenHashAsync(hash, now)`, `RevokeAsync(hash, at)`, `DeleteEndedAsync(now)`

Rules every repository follows:

- Returns entities when the caller will change them (tracked), and `AsNoTracking` entities or small read records (such as `HistoryPage`) for reads. Never `IQueryable`.
- Takes `customerId` on every customer-owned lookup, so a missing or foreign id returns `null` in one place.

- `Add…` methods stage changes only; the service decides when to save.
- Accepts a `CancellationToken` on every async method.

Transfers, quotes and idempotency records sit in one repository because they are only ever used together inside the transfer flow. `ICustomerRepository` and `ISessionRepository` are added beyond the four domain repositories so that `AuthController` and `SessionService` also stop using `BankDbContext` directly.

Unit of work

`BankDbContext` itself is the unit of work. Services may use it only for `Database.BeginTransactionAsync`, `SaveChangesAsync` and `ChangeTracker.Clear()`, and call `SaveChangesAsync` once per operation. Repositories share the same scoped context instance, so everything staged during a transfer commits together. A separate `IUnitOfWork` wrapper is not worth it here: it would only forward these three calls.

Registration

All repositories and services are registered as scoped in `Program.cs` through one `AddBankServices()` extension, so a request gets one `BankDbContext` shared by its services and repositories. `TransferValidator` is stateless and can be a singleton. `CleanupService` stays a hosted service and creates a scope per run.

Refactoring safely

The 29 HTTP integration tests are the safety net: they exercise the public contract, so they must stay green without changes at every step.

1. Accounts and payees: add `IAccountRepository`, `ILedgerRepository` reads, `IPayeeRepository`, `AccountService` and `PayeeService`; slim the two controllers.
2. Sign-in: add `ICustomerRepository`, `ISessionRepository` and `AuthService`; slim `AuthController`.
3. Transfers: move queries and raw SQL from `TransferService` into `ITransferRepository` and `IAccountRepository`; fold `LedgerService` into `ILedgerRepository.PostEntryAsync`.
4. Lock it in: add an architecture test that fails if any type in `Controllers` depends on `BankDbContext` or a repository, or any service depends on `IQueryable`.
5. Optionally add unit tests for `TransferService` against fake repositories, which the interfaces now make possible.

Status (2026-10-25): steps 1 to 4 are implemented. The 29 existing tests pass unchanged, plus 3 new architecture tests. Step 5 is not started. Set-based deletes and the session revoke (`ExecuteDeleteAsync` , `ExecuteUpdateAsync`) apply immediately rather than waiting for `SaveChangesAsync` ; their interface comments say so.

ASP.NET Core implementation notes

The API is one ASP.NET Core Web API project on .NET 10 with controllers, EF Core, and a test project; the examples in this doc follow Alice's seeded data through one 200.00 transfer from checking to savings.

Concern	Choice
Projects	`OnlineBankingAPI` (Web API) and `OnlineBankingAPI.Tests` (xUnit)
Folders	`Controllers` , `Contracts` (request/response records), `Domain` , `Data` (DbContext, configurations, migrations, seeder), `Services` (Auth, Session, Account, Payee, Transfer, TransferValidator), Repositories (one interface plus EF Core class per area; see Layered architecture)
Data access	EF Core with the Npgsql provider (PostgreSQL 17, run locally in Docker), migrations checked in; LINQ and raw SQL live only in repositories
Auth	Custom `AuthenticationHandler` that reads the cookie or bearer token, hashes it and loads the session; fallback policy requires an authenticated user
Passwords	`PasswordHasher<Customer>` from `Microsoft.Extensions.Identity.Core`
Validation	Data annotations for shape, `TransferValidator` service for business rules returning rule codes
Errors	`AddProblemDetails` , `UseExceptionHandler` , and a mapper from rule codes to Problem Details

Concern	Choice
Idempotency	An endpoint filter on `POST /transfers` that reads the header and checks the record; the write happens inside `TransferService`'s transaction
Transactions	`IsolationLevel.ReadCommitted` with `SELECT … FOR UPDATE` row locks taken in ascending id order
Rate limiting	Built-in `AddRateLimiter`: fixed window on login, token bucket per session
OpenAPI	`Microsoft.AspNetCore.OpenApi`, served at `/openapi/v1.json`, with Scalar UI in Development
Time	Inject `TimeProvider` so quote expiry and session timeouts are testable

Tests that prove the requirements

- Customer A requesting customer B's account, transactions or payee gets `404`.
- After logout, the same token gets `401` on every protected endpoint.
- Twenty parallel `POST /transfers` calls with the same key create exactly one transfer and one pair of ledger entries; all twenty responses are identical.
- The same quote submitted with two different keys creates one transfer and one `409 quote_already_used`.
- Every validation rule in the Transfers table has a test returning its code.
- After any transfer, both accounts' `Balance` equals their last `BalanceAfter`, and the debit and credit amounts match.
- History paging over 320 entries returns every entry exactly once while a transfer posts mid-scroll.

Open questions

- Bank time zone for date filters: assumed US Eastern.
- Payee management: read-only now; add create and delete later?